

CI/CD Pipeline Architecture and Status

CI Phase

0. New Merge Request

1. Merge Request Approval

- Trigger point for the automated GitHub Actions pipeline

2. Linting Checks

- We are using ESLint and PyLint for the initial linting steps
 - Tech Stack: ESLint(Frontend), PyLint(Backend)
- **Status: This setup has been completed**

3. Build

- This stage includes building the docker image and running the build of the project.
 - Dependency Build:
 - 'name: Build Docker image run: docker build -t my-app'
 - Run the Build:
 - 'name: Run build inside container run: docker run --rm my-app npm run build'
- These commands are executed by a Github Action Cloud Runner that creates a temporary VM and tests the build
- **Status: This setup has been completed**

4. Testing

- There are two types of testing that will run in parallel. Both testings must pass before moving to the next step.
 - Unit Testing
 - This phase will run specific feature focused unit tests. These tests will be for individual components and should run very quick. No external dependencies.
 - Tech Stack: Jest, PyTest
 - Integration Testing
 - This phase will test to see if the different components of the project work together as a group. This test will include calls from the database and APIs to ensure that the integrations are working as intended.
 - Tech Stack: Docker Compose, Postman, Custom Shell Scripts
- **Status: This is still in progress**

5. Code Review Approval

- Standard manual code review process. This is the gate between the CI and CD in the pipeline

CD Phase

6. Build Container Image

- This phase packages the application and all dependencies into a portable container. This guarantees consistency across all environments and eliminates dependency issues.
- Tech Stack: GitHub Actions, DockerHub
- **Status: The DockerHub setup is ready, however we still need to test the initial runs for this.**

7. Push Container to Registry

- This step uploads the built container image to a centralized repository with version tags. This is so that we have a single source of truth for the deployed version and enables rollback to previous version.
- Tech Stack: DockerHub
- **Status: The DockerHub setup is ready, however we still need to test the initial runs for this.**

8. Deploy to Staging

- This step deploys the container to a production-like environment for final validation.
- Tech Stack: GCP Compute Engine, Custom Shell Scripts
- **Status: We have setup the GCP compute engine VM. We are setting up ports running, one for validation and one for production.**

9. Validation and Smoke Tests

- This step catches deployment-specific issues like configuration errors, broken endpoints, or environment problems before production. It also allows people to use the website and scout for any Prod UI/UX issues.
- Tech Stack: Custom Health Check Scripts, Curl Scripts
- **Status: This is still in progress. We have not wrote the health check scripts yet.**

10. Deploy to Production

- This step releases the validated container to the live production environment serving real users.
- Tech Stack: GCP Compute Engine
- **Status: This step has been completed.**

CI/CD Diagram

